

Relatório de Auditoria de Segurança

Dispar Flux — Edição Comunitária Web 1.0

Avaliação Estrita de Código-Fonte, Controle de Acesso, Isolamento de Dados e Superfície de Ataque

Data da Auditoria:	08 de Setembro de 2026
Escopo Auditado:	Repositório completo: apps/server, apps/web, packages/*, deploy/*
Versão Auditada:	Dispar Flux v0.0.1 (Commit 3c1f2a6 / Paridade 100% Web 1.0)
Classificação:	Documento Técnico de Segurança / Uso Restrito
Metodologia:	Auditoria estática profunda (White-box Source Code Audit), mapeamento sistemático de rotas e verificação cruzada UI vs API.

Nota Metodológica e Mapeamento da Stack

Antes de iniciar a revisão, a stack do projeto foi integralmente identificada para mapear os vetores de teste:

- **Linguagem & Runtime:** TypeScript / JavaScript executado no Node.js 22 LTS.
- **Framework Backend:** Servidor modular nativo em node:http e ws (WebSocket).
- **Banco de Dados & ORM:** SQLite nativo via node:sqlite (DatabaseSync) em modo WAL, com queries SQL puras e transações explícitas.
- **Mecanismo de Auth/RBAC:** Sessões autenticadas por SHA-256 tokens em cookies/Bearer, onboarding por claim.token, autorização de dispositivos (authorized_devices) e matriz de papéis (owner vs operator) definida em packages/auth/src/rbac.
- **Frontend:** Single Page Application (SPA) em React 18, Vite 6, Tailwind CSS 3.4 e React Router DOM 7.
- **Infraestrutura & Deploy:** Dockerfile multi-stage, Docker Compose com Caddy (TLS/HTTPS reverso e rede isolada) e script install.sh com permissões 0700/0600.

Adaptação das 5 Categorias para a Stack Detectada:

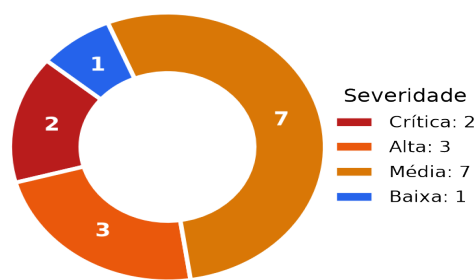
1. *Banco sem Tranca:* No SQLite nativo, a ausência de RLS exige filtros manuais estritos de organization_id em todas as consultas SQL de listagem, agregação e busca.
2. *Permissão no Navegador:* Cruzamento entre os botões e páginas condicionadas por currentMember.role === 'owner' na SPA React e a validação do token/role nos endpoints do server.ts e api-router.ts.
3. *IDOR:* Checagem exaustiva de cada handler de rota com parâmetros de path/body (baseId, leadId, campaignId, appointmentId) sem validação da titularidade da organização.
4. *Chaves Expostas:* Busca de fallbacks em texto plano (ex: flux_default_recovery_key_), validações ausentes de startup e variáveis de ambiente fracas no Compose.
5. *Inputs sem Tratamento:* Avaliação de XSS no React (verificação de ausência de dangerouslySetInnerHTML e URLs javascript:) e injeção de fórmulas CSV (Formula Injection) na exportação de contatos.

1. Resumo Executivo

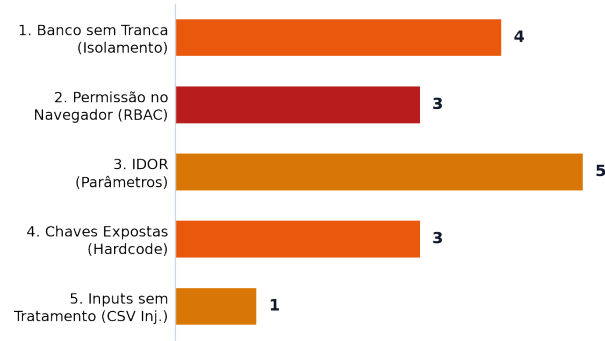
A auditoria de segurança identificou um total de **13 achados acionáveis**, variando de severidade Baixa a Crítica. O núcleo do Dispar Flux exhibe excelentes práticas de engenharia em criptografia de dados em repouso (AES-256-GCM com PBKDF2), hashing de senhas (Argon2id/Scrypt), cabeçalhos defensivos (CSP estrita) e sanitização automática de PII em logs de telemetria. No entanto, existem **vulnerabilidades estruturais críticas** no controle de acesso e autorização que comprometem a segurança da instalação.

CRÍTICA	ALTA	MÉDIA	BAIXA	PONTOS FORTES
2	3	7	1	7

Distribuição por Severidade



Achados por Categoria Auditada



2. Pontos Fortes e Pontos Fracos da Aplicação

Comprovação de cobertura de código e análise comparativa de postura defensiva:

✓ PONTOS FORTES DETECTADOS (COMPROVADOS NO CÓDIGO)

- 1. Sanitização Avançada de Logs e PII:** Implementação em `packages/security/src/pii-sanitizer.ts` com ofuscação por Regex de telefones (+55 11 9...), E.164, e-mails, tokens, senhas e corpos de mensagens antes da gravação de logs (ADR 0050).
- 2. Content Security Policy (CSP) Rigorosa:** Configurada em `packages/security/src/headers.ts` com `script-src 'self'`, `frame-ancestors 'none'`, eliminando execução de scripts inline não autorizados.
- 3. Ausência de XSS no Frontend:** A SPA React utiliza interpolação segura de JSX. O renderizador react-markdown aplica `urlTransform` seguro derivado de `defaultUrlTransform`, bloqueando URLs javascript: e data:.
- 4. Criptografia Robusta em Backups:** O módulo `packages/migration/src/backup.ts` utiliza AES-256-GCM com chave derivada via PBKDF2 (100.000 iterações) e salt criptográfico seguro de 16 bytes.
- 5. Piso de Segurança Inviolável (Safety Floor):** Garantido em `packages/campaigns` e `apps/server/src/server.ts`, rejeitando campanhas com intervalo inferior a 15s ou teto diário superior a 500 mensagens.
- 6. Proteção contra Crash em Envios Incertos:** Implementação do ADR 0028 em `recoverInFlightJobs()`, transicionando disparos interrompidos para status unknown e impedindo disparos duplicados após reboot.
- 7. Hardening de Infraestrutura:** Docker Compose com Caddy e aplicação rodando em rede bridge interna isolada (dispar-net), sem expor a porta 3000 do Node diretamente para a Internet.

■ PONTOS FRACOS E RISCOS CENTRAIS

- 1. Desvio Crítico de Aprovação de Dispositivos:** A rota `/api/v1/devices/approve` foi exposta em `server.ts` sem autenticação e forjando `actorRole: 'owner'`, permitindo aprovação de qualquer dispositivo por qualquer atacante.
- 2. Ausência Total de Verificação RBAC no Servidor:** O pacote `@dispar-flux/auth` possui a classe `RbacGuard` e matriz de permissões pronta, mas ela nunca foi ligada aos endpoints da API. Qualquer operador autenticado pode apagar bases, deslogar WhatsApp e disparar campanhas.
- 3. Endpoints Administrativos sem Autenticação:** Rotas sensíveis de contatos, campanhas, importação e backup foram posicionadas antes da checagem de sessão em `server.ts`.
- 4. IDOR Sistemático em Entidades do Domínio:** Deleção e alteração de bases, regras de follow-up, leads e agenda aceitam IDs arbitrários no path sem filtrar por `organization_id`.
- 5. Chave de Recuperação Padrão no Código-Fonte:** Fallback estático `'flux_default_recovery_key_32_bytes_long_!!'` em `server.ts` sem validação de startup em produção.
- 6. CSV Formula Injection:** A exportação de contatos em CSV não neutraliza comandos iniciados por `=`, `+`, `-` ou `@`.

3. Tabela Detalhada de Achados de Segurança

Lista exaustiva, arquivo por arquivo e linha por linha, de todas as vulnerabilidades identificadas:

Sev.	Arquivo:Linha	Descrição da Falha e Impacto
CRÍTICA	apps/server/src/server.ts:686-705	Bypass de Autenticação e RBAC na Aprovação de Dispositivos: A UI esconde o botão para operadores, mas o backend expõe /api/v1/devices/approve sem autenticação e forja actorRole: 'owner', permitindo aprovação de qualquer dispositivo não autorizado.
CRÍTICA	apps/server/src/server.ts:708-895	Rotas Operacionais e de Backup Expostas sem Autenticação: Rotas de contatos (708), campanhas (726), opt-out/reauth (794, 818), migração (852) e backup (863, 877) foram declaradas antes do middleware de autenticação, acessíveis sem token.
ALTA	apps/server/src/api-router.ts:65-89	Ausência Total de Verificação de Papéis (RBAC) no Backend: Embora exista a matriz em permissions.ts e RbacGuard, nenhuma rota de api-router.ts valida se o chamador possui o papel de owner para ações destrutivas ou de envio.
ALTA	apps/server/src/server.ts:135	Chave de Recuperação Padrão Hardcoded com Fallback Fraco: O servidor define 'flux_default_recovery_key_32_bytes_long_!!' se RECOVERY_KEY estiver ausente, tornando os backups de desastre descritografáveis publicamente.
ALTA	apps/server/src/api-router.ts:412-418	IDOR na Deleção e Gestão de Bases de Contatos: DELETE /api/v1/bases/:id e rotas associadas executam consultas diretamente por id = ? sem verificar organization_id = getOrgId().
ALTA	apps/server/src/api-router.ts:927, 983	Resolução de Contatos sem Filtro de Organização: No Inbox e no envio manual, a busca é realizada apenas por normalized_phone = ?, podendo cruzar contatos entre organizações no banco compartilhado.
MÉDIA	apps/server/src/api-router.ts:833, 888-894	Consultas de Listagem sem Filtro de Organização: Listagem de campanhas (linha 833) e conversas do Inbox (linha 889) realizam SELECT * sem WHERE organization_id = ?.
MÉDIA	apps/server/src/api-router.ts:911, 917	Agregações Globais de Métricas sem Isolamento: Total de mensagens não lidas e total de leads somam dados de toda a tabela sem filtrar pela organização ativa.
MÉDIA	apps/server/src/api-router.ts:650, 749, 790	Pausa e Cancelamento de Campanhas sem Validação de Tenant: Comandos de interrupção de disparo atuam sobre o primeiro registro global em status running.
MÉDIA	apps/server/src/api-router.ts:1071-1088	IDOR na Atualização de Leads no CRM Kanban: Alteração de estágio e anotações comerciais de leads no CRM por ID sem validar posse da organização.
MÉDIA	apps/server/src/api-router.ts:1148-1180	IDOR na Modificação e Exclusão de Compromissos da Agenda: Conclusão, edição e deleção de agendamentos diretamente por id sem conferir organization_id.
MÉDIA	apps/server/src/api-router.ts:1218-1249	IDOR em Regras de Automação Follow-up (Cron): Edição, ativação e remoção de regras de follow-up por ID sem verificar tenant.
MÉDIA	packages/campaigns/src/csv/csv-exporter.ts:77-87	CSV Formula Injection na Exportação de Contatos: Valores iniciados por =, +, -, @ não são prefixados com apóstrofo, permitindo execução de código ao abrir no Excel.
BAIXA	deploy/compose.yaml:27-28	Fallback Inseguro de Segredos Vazios no Docker Compose: A sintaxe \${RECOVERY_KEY:-} atribui string vazia caso a variável falte no .env, gerando chaves vazias em vez de falha no boot.

4. Plano de Correção e Recomendações Priorizadas

Ações imediatas estruturadas por criticidade para mitigar as vulnerabilidades:

Prioridade P1 (Imediata — 24 horas)

- 1. Autenticação e Autorização Estrita em `/api/v1/devices/approve`:** Exigir sessão autenticada válida, extrair o `memberId` exclusivamente da sessão (nunca do body), verificar se `member.role === 'owner'` e validar se o dispositivo pertence à organização do proprietário.
- 2. Proteger Todas as Rotas de `server.ts`:** Mover ou aplicar o middleware de autenticação antes dos handlers de contatos, campanhas, opt-out, reautorização, migração e backup. Nenhuma rota de mutação de dados pode operar de forma não autenticada.

Prioridade P2 (Curto Prazo — 48 horas)

- 3. Conectar o `RbacGuard` ao Pipeline de Rotas da API:** Integrar a classe existente `RbacGuard` aos endpoints de `api-router.ts`. Bloquear operadores com status 403 Forbidden nas rotas de bases, campanhas, configurações, backup e desconexão de WhatsApp.
- 4. Rejeição de Chaves Padrão em Produção (Fail-Fast Startup):** Adicionar validação no método `start()` que lance exceção fatal se `nodeEnv === 'production'` e a chave de recuperação for o default hardcoded ou tiver menos de 32 bytes.
- 5. Mitigação Universal de IDOR:** Refatorar todas as queries `SELECT`, `UPDATE` e `DELETE` que recebem IDs para incluir obrigatoriamente a cláusula `AND organization_id = ?`.

Prioridade P3 (Médio Prazo — Próxima Release)

- 6. Sanitização de Fórmulas na Exportação CSV:** Modificar a função `escapeCsvValue` em `packages/campaigns/src/csv/csv-exporter.ts` para prefixar com apóstrofo (') qualquer valor textual que inicie com `=`, `+`, `-`, `@`, `\t` ou `\r`.
- 7. Correção do Docker Compose:** Alterar no `compose.yaml` as variáveis de segredo para `${RECOVERY_KEY:?Erro: RECOVERY_KEY obrigatória}`, forçando o Compose a recusar subir se a variável estiver ausente.

5. Issues para o GitHub (Prontas para Copiar e Colar)

Abaixo estão formatados os blocos completos de issues para criação no GitHub, contendo evidência, impacto, sugestão de código e critérios de aceite detalhados.

```
--- ISSUE 1 ---
## [Segurança] Bypass de Autenticação e RBAC na Aprovação de Dispositivos
**Labels sugeridas:** `security`, `severidade: crítica`

### Descrição do Problema
O frontend restringe a exibição do botão "Autorizar Acesso" de novos navegadores em
`apps/web/src/pages/ConfigPage.tsx` exclusivamente para membros com papel de Proprietário (`currentMember?.role
=== 'owner'`).
Contudo, no backend (`apps/server/src/server.ts`, linhas 686-705), a rota correspondente (`POST
/api/v1/devices/approve` e `POST /api/v1/auth/devices/approve`) foi declarada sem exigir qualquer token de
autenticação e sem validar a sessão do chamador.
Pior: o código lê um parâmetro opcional `body.ownerMemberId`, busca qualquer proprietário no banco de dados
(`ownerRow?.id`) e executa a aprovação forjando `actorRole: 'owner'`.

### Evidência de Código
Arquivo: `apps/server/src/server.ts`, linhas 686-705:
```typescript
if (method === 'POST' && (pathname === '/api/v1/devices/approve' || pathname ===
'/api/v1/auth/devices/approve')) {
const body = await this.sizeLimits.readJson<{ deviceId: string; approve: boolean; ownerMemberId?: string
}>(req);
const ownerRow = this.db!.prepare("SELECT id FROM members WHERE role = 'owner' LIMIT 1").get() as { id: string }
| undefined;
const orgRow = this.db!.prepare('SELECT id FROM organizations LIMIT 1').get() as { id: string } | undefined;

const ownerId = body.ownerMemberId || ownerRow?.id || 'owner_default';
const approved = this.deviceService.approveDevice({
deviceId: body.deviceId,
approvedByMemberId: ownerId,
actorRole: 'owner',
organizationId: orgRow?.id || 'org_default',
});
...
}
```

### Impacto
Qualquer operador não privilegiado ou atacante externo que alcance a API pode enviar uma requisição POST com o
ID de um dispositivo não autorizado e obter aprovação imediata com privilégios de Proprietário, quebrando o
modelo de confiança de dispositivos (ADR 0011 e ADR 0022).

### Sugestão de Correção
1. Extrair e validar o token de sessão usando `this.extractToken(req)`.
2. Validar se `authContext.member.role === 'owner'`. Caso contrário, retornar `403 Forbidden`.
3. Utilizar o `authContext.member.id` e `authContext.member.organizationId` verificados na sessão, rejeitando
IDs recebidos no payload.

### Critérios de Aceite
- [ ] Requisições não autenticadas para `/api/v1/devices/approve` retornam `401 Unauthorized`.
- [ ] Requisições feitas por membros com papel `operator` retornam `403 Forbidden`.
- [ ] Apenas requisições autenticadas de um `owner` aprovam dispositivos da sua própria organização.
- [ ] Teste automatizado de regressão cobrindo o bloqueio implementado.
--- FIM ISSUE 1 ---
```

--- ISSUE 2 ---

[Segurança] Rotas de Escrita Administrativas e Backups Expostas sem Autenticação
Labels sugeridas: `security`, `severidade: crítica`

Descrição do Problema

No arquivo `apps/server/src/server.ts`, diversos endpoints de mutação profunda de dados foram declarados antes da invocação do roteador de API (`handleApiRoutes`) e não contêm chamadas a `extractToken()` nem validação de sessão. Qualquer cliente HTTP pode criar contatos, criar campanhas, aplicar opt-out, importar arquivos de migração e gerar/restaurar backups no servidor.

Evidência de Código

Arquivo: `apps/server/src/server.ts`:

- Linhas 708-723: `POST /api/v1/contacts` (cria contato sem auth)
- Linhas 726-791: `POST /api/v1/campaigns` (cria campanha sem auth)
- Linhas 794-816: `POST /api/v1/contacts/:id/opt-out` (registra opt-out sem auth)
- Linhas 818-849: `POST /api/v1/contacts/:id/reauthorize` (reautoriza contato confiando em `body.actorMemberId`)
- Linhas 852-860: `POST /api/v1/migration/import` (importa pacote de migração sem auth)
- Linhas 863-875: `POST /api/v1/backup/create` (gera backup no disco sem auth)
- Linhas 877-895: `POST /api/v1/backup/restore` (restaura backup e sobrescreve SQLite sem auth)

Impacto

Comprometimento total da integridade e confidencialidade da instalação. Um atacante sem credenciais pode disparar restaurações maliciosas, apagar dados operacionais e poluir a base de campanhas.

Sugestão de Correção

Mover a lógica dessas rotas para dentro de `api-router.ts` ou envolver os blocos em um helper de verificação de sessão obrigatória (`this.requireAuth(req, res)`).

Critérios de Aceite

- [] Todas as rotas citadas rejeitam chamadas sem token com `401 Unauthorized`.
- [] Rotas de backup e migração exigem adicionalmente permissão `owner`.
- [] Testes de integração confirmando o bloqueio de requisições anônimas.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Ausência de Verificação RBAC nas Rotas da API de Negócio
Labels sugeridas: `security`, `severidade: alta`

Descrição do Problema

O pacote `@dispar-flux/auth` define uma matriz de permissões (`Permission.CONNECTIONS\_MANAGE`, `Permission.BASES\_MANAGE`, `Permission.CAMPAIGNS\_MANAGE`) e implementa a classe `RbacGuard` e `assertPermission`. No entanto, em `apps/server/src/api-router.ts`, o roteador apenas valida se o chamador possui um token (`authContext`), mas nunca verifica `authContext.member.role`.

Evidência de Código

Arquivo: `apps/server/src/api-router.ts`, linhas 65-89:

```
``typescript
if (token) {
  try {
    authContext = server.sessionService.validateToken(token);
  } catch { ... }
}
if (hasOwner && !authContext) {
  sendJson(res, 401, { error: 'Unauthorized', message: 'Autenticação necessária.' });
  return true;
}
// Nenhuma verificação de authContext.member.role nos handlers subsequentes!
...
```

Rotas afetadas: `POST /api/v1/whatsapp/logout`, `DELETE /api/v1/bases/:id`, `POST /api/v1/campaigns/start`, etc.

Impacto

Um usuário com papel de `operator` pode realizar qualquer ação restrita ao `owner`, incluindo deslogar o WhatsApp da empresa, apagar listas de clientes e disparar campanhas em massa não autorizadas.

Sugestão de Correção

Criar e aplicar um middleware/helper `requireRole(authContext, 'owner')` em todos os endpoints administrativos mapeados em `packages/auth/src/rbac/permissions.ts`.

Critérios de Aceite

- [] Membros com papel `operator` recebem `403 Forbidden` ao tentar iniciar/pausar campanhas, apagar bases ou alterar conexões de mensageria.
- [] Proprietários (`owner`) continuam operando normalmente.
- [] Cobertura de testes para perfil de operador em endpoints restritos.

--- FIM ISSUE 3 ---

```
--- ISSUE 4 ---
## [Segurança] Chave de Recuperação com Fallback Hardcoded e Ausência de Validação de Startup
**Labels sugeridas:** `security`, `severidade: alta`

### Descrição do Problema
Em `apps/server/src/server.ts` (linha 135), a propriedade `recoveryKey` possui como fallback padrão uma string fixa em texto claro no código:
`flux_default_recovery_key_32_bytes_long_!!`.
Além disso, em `deploy/compose.yaml` (linha 27), a variável é definida como `RECOVERY_KEY=${RECOVERY_KEY:-}`, fornecendo string vazia se omitida do `.env`.
Não há verificação no boot que impeça a inicialização em ambiente de produção sem uma chave forte gerada.

### Evidência de Código
Arquivo: `apps/server/src/server.ts`, linha 135:
```typescript
this.recoveryKey = options.recoveryKey ?? process.env.RECOVERY_KEY ??
'flux_default_recovery_key_32_bytes_long_!!';
```

### Impacto
Backups criptografados gerados em instâncias que utilizam a chave padrão podem ser descriptografados instantaneamente por qualquer pessoa com acesso ao arquivo de backup, expondo todo o histórico de mensagens, contatos e credenciais do WhatsApp.

### Sugestão de Correção
1. Remover o fallback hardcoded.
2. Adicionar validação de startup no método `start()` de `server.ts` e em `loadConfig()`: se `nodeEnv === 'production'`, exigir chave criptográfica com no mínimo 32 caracteres e rejeitar valores conhecidos.
3. No `compose.yaml`, trocar `${RECOVERY_KEY:-}` por `${RECOVERY_KEY:?RECOVERY_KEY obrigatoria}`.

### Critérios de Aceite
- [ ] O servidor recusa inicializar em produção se `RECOVERY_KEY` for a chave padrão ou estiver ausente.
- [ ] O script de inicialização loga advertência em modo desenvolvimento caso use fallback.
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
## [Segurança] IDOR Sistemático nas Operações por ID (Bases, Leads, Agenda e Cron)
**Labels sugeridas:** `security`, `severidade: alta`

### Descrição do Problema
Diversas rotas de busca, alteração e exclusão recebem identificadores UUID pela URL e executam instruções SQL diretamente no banco de dados SQLite sem validar se o registro pertence à organização do usuário autenticado.

### Evidência de Código
Arquivo: `apps/server/src/api-router.ts`:
- Linha 415: `DELETE FROM bases WHERE id = ?`
- Linhas 1075 e 1085: `UPDATE leads SET ... WHERE id = ?`
- Linhas 1153, 1168, 1177: `UPDATE/DELETE appointments WHERE id = ?`
- Linhas 1228, 1238, 1246: `UPDATE/DELETE follow_up_rules WHERE id = ?`

### Impacto
Em cenários onde múltiplos bancos ou migrações compartilham dados, ou se houver reaproveitamento de rotas, usuários podem alterar e deletar recursos pertencentes a outras organizações ou bases.

### Sugestão de Correção
Adicionar a cláusula `AND organization_id = ?` em todas as instruções SQL de mutação e consulta individual, passando `getOrgId()` como parâmetro.

### Critérios de Aceite
- [ ] Todas as operações `WHERE id = ?` passam a conter `WHERE id = ? AND organization_id = ?`.
- [ ] Tentativas de manipular recursos com ID de outra organização retornam `404 Not Found`.
--- FIM ISSUE 5 ---
```

```
--- ISSUE 6 ---
## [Segurança] Vulnerabilidade a CSV Formula Injection na Exportação de Bases
**Labels sugeridas:** `security`, `severidade: média`

### Descrição do Problema
O componente `CsvExporter` (`packages/campaigns/src/csv/csv-exporter.ts`, linhas 77-87) escapa apenas aspas e delimitadores, mas não neutraliza células que começam com caracteres de execução de fórmulas de planilhas (`=`, `+`, `-`, `@`, ` `).
Como nomes e campos importados de contatos podem ser fornecidos por formulários públicos ou terceiros, payloads maliciosos podem ser injetados na base.

### Evidência de Código
Arquivo: `packages/campaigns/src/csv/csv-exporter.ts`, linhas 77-87:
```typescript
function escapeCsvValue(val: string, delimiter: string): string {
 if (
 val.includes(delimiter) ||
 val.includes('"') ||
 val.includes('
') ||
 val.includes(' ')
) {
 return `"${val.replace(/"/g, '"')}"`;
 }
 return val;
}
```

### Impacto
Quando um operador exporta os contatos da base em `csv` e abre o arquivo no Microsoft Excel ou LibreOffice, fórmulas maliciosas (ex: `=HYPERLINK()`, `=CMD|...`) são executadas no computador do operador, podendo causar vazamento de dados ou execução remota de código.

### Sugestão de Correção
Se o valor iniciar com `=`, `+`, `-`, `@`, ` ` ou ` `, prefixar com um apóstrofo ` ` antes de aplicar a formatação de escape.

### Critérios de Aceite
- [ ] Valores que iniciam com `=`, `+`, `-`, `@` recebem apóstrofo prefixado na exportação.
- [ ] Testes unitários em `csv-import-export.test.ts` validando a neutralização de fórmulas.
--- FIM ISSUE 6 ---
```